CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Artificial Intelligence & Machine Learning: Campus Master Edition (Prescribed Textbook)

Subject Track: Artificial Intelligence & Machine Learning (AI / ML)

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Authoritative campus placement textbook covering Artificial Intelligence and Machine Learning from mathematical foundations (gradient descent, loss functions) to supervised/unsupervised learning, Deep Learning, CNNs, Transformers, LLMs, and MLOps.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for Artificial Intelligence & Machine Learning (AI / ML) on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: Foundations of AI, Machine Learning Paradigms & Mathematics

Ch 2: Supervised Learning: Regression & Classification

Ch 3: Unsupervised Learning & Dimensionality Reduction

Ch 4: Model Evaluation, Cross-Validation & The Bias-Variance Tradeoff

Ch 5: Neural Networks & Deep Learning Foundations

Ch 6: Convolutional Neural Networks & Recurrent Networks

Ch 7: Natural Language Processing & Transformers Architecture

Ch 8: Solved AI/ML Placement Interview Case Studies & System Design

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: Artificial Intelligence & Machine Learning (AI / ML)

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Authoritative campus placement textbook covering Artificial Intelligence and Machine Learning from mathematical foundations (gradient descent, loss functions) to supervised/unsupervised learning, Deep Learning, CNNs, Transformers, LLMs, and MLOps.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: Foundations of AI, Machine Learning Paradigms & Mathematics](#chapter-1)

- [Chapter 2: Supervised Learning: Regression & Classification](#chapter-2)
- [Chapter 3: Unsupervised Learning & Dimensionality Reduction](#chapter-3)
- [Chapter 4: Model Evaluation, Cross-Validation & The Bias-Variance Tradeoff](#chapter-4)

- [Chapter 5: Neural Networks & Deep Learning Foundations](#chapter-5)
- [Chapter 6: Convolutional Neural Networks & Recurrent Networks](#chapter-6)
- [Chapter 7: Natural Language Processing & Transformers Architecture](#chapter-7)
- [Chapter 8: Solved AI/ML Placement Interview Case Studies & System Design](#chapter-8)

CHAPTER 1

Foundations of AI, Machine Learning Paradigms & Mathematics

Key Principle / Exam Takeaway: Master Supervised vs Unsupervised vs Reinforcement Learning, Linear Algebra vectors/matrices, Gradient Descent optimization, and Convex Loss functions.

Core Curriculum Topics Covered in this Chapter:

- AI vs Machine Learning vs Deep Learning vs Generative AI taxonomy
- Paradigms: Supervised (labels), Unsupervised (patterns), Reinforcement Learning (reward/penalty agents)
- Linear Algebra essentials: Dot products, matrix multiplication, eigenvectors, and eigenvalues
- Calculus foundations: Partial derivatives, gradients, the Jacobian, and Hessian matrices
- Optimization: Cost functions (MSE, Cross-Entropy) and Gradient Descent (Batch, Mini-Batch, Stochastic)

· 1. Core Concepts & In-Depth Explanation

Artificial Intelligence (AI) simulates human cognitive decision-making in machines. Machine Learning (ML) is the subset of AI focused on algorithms that learn statistical patterns directly from data without being explicitly rule-programmed.

Machine Learning Paradigms: 1. **Supervised Learning:** Training data includes features X and ground-truth target labels y. Tasks: Regression (continuous output) and Classification (discrete categorical output). 2. **Unsupervised Learning:** Unlabeled features X. Identifies inherent latent groupings (Clustering) or compresses feature spaces (Dimensionality Reduction). 3. **Reinforcement Learning:** An autonomous agent learns an optimal policy through trial-and-error interactions with an environment, maximizing cumulative scalar rewards.

Gradient Descent: The universal engine optimizing model weights W by iteratively stepping in the opposite direction of the loss function gradient: $$W_{t+1} = W_t - \alpha \nabla L(W_t)$$ Where α is the learning rate. Mini-batch Gradient Descent balances vectorization speed on GPUs with stochastic noise that helps escape shallow local minima.

· 2. Key Formulas, Rules & Complexity Classes

1. **Gradient Descent Weight Update:** $$W := W - \alpha \frac{\partial L}{\partial W}$$ 2. **Mean Squared Error (MSE Loss for Regression):** $$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$ 3. **Binary Cross-Entropy Loss (BCE for Classification):** $$L_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \Big[y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \Big]$$

· 3. Step-by-Step Solved Examples

Example 1.1: Gradient Descent Optimization Step Trace

- **Difficulty / Target Companies:** `Easy | Google, Amazon, TCS Digital`

- **Problem Statement:** Given loss function $L(w) = w^2 - 4w + 5$ and learning rate $\alpha = 0.1$. Starting from initial weight $w_0 = 0$, compute w_1 and w_2.

Step-by-Step Walkthrough & Logic:

- Step 1: Compute derivative: $\frac{dL}{dw} = 2w - 4$.
- Step 2: At $w_0 = 0$: Gradient $= 2(0) - 4 = -4$.
- Step 3: Update $w_1 = w_0 - \alpha \times (\text{Gradient}) = 0 - (0.1 \times -4) = 0 + 0.4 = 0.4$.
- Step 4: At $w_1 = 0.4$: Gradient $= 2(0.4) - 4 = 0.8 - 4 = -3.2$.
- Step 5: Update $w_2 = 0.4 - (0.1 \times -3.2) = 0.4 + 0.32 = 0.72$ (converging steadily towards minimum at $w = 2$).

Complete Solved Code / Calculation:

```
# Gradient Descent Convergence:
# w0 = 0
# w1 = 0.4
# w2 = 0.72
# w* = 2.0 (Global Minimum where derivative = 0)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates how negative gradients drive parameters toward the global cost minimum.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: What happens if the learning rate α in gradient descent is set too high?

Options:

- A) Model converges instantly
- B) Loss oscillates wildly and diverges without finding the minimum
- C) Weights freeze at zero
- D) Prevents overfitting

Answer: **B) Loss oscillates wildly and diverges without finding the minimum** **Detailed Solution & Reasoning:** An excessively large learning rate overshoots the minimum valley, causing numerical divergence.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Google)

- **Interview Question:** _What is the difference between Stochastic Gradient Descent (SGD), Batch Gradient Descent, and Mini-batch Gradient Descent?_
- **Model High-Scoring Answer:** **Batch Gradient Descent** computes the gradient across the entire training dataset before taking a single weight step; it is mathematically stable but computationally slow and intractable for massive datasets. **Stochastic Gradient Descent (SGD)** computes the gradient and updates weights after *every single individual sample*; it is fast but introduces high stochastic variance and noisy oscillations. **Mini-batch Gradient Descent** is the standard industry compromise: it computes gradients over batches of 32 to 512 samples, leveraging GPU SIMD vectorization while providing smooth, robust convergence.

- **Interviewer Follow-Up & Pro Tip:** ? Cite hardware GPU tensor parallelism as the reason mini-batches are standard.

CHAPTER 2

Supervised Learning: Regression & Classification

Key Principle / Exam Takeaway: Master Ordinary Least Squares (OLS), Logistic Regression (Sigmoid / Odds ratio), Decision Trees (Gini vs Entropy), Random Forests, and Support Vector Machines (SVM).

Core Curriculum Topics Covered in this Chapter:

- Linear Regression: OLS assumptions, normal equation vs gradient descent, R-squared evaluation
- Regularization: Ridge (L2 penalty) vs Lasso (L1 feature selection penalty)
- Logistic Regression: Log-odds (Logit), Sigmoid activation, decision thresholding
- Decision Trees: Information Gain (Shannon Entropy) vs Gini Impurity, pruning
- Ensemble methods: Bagging (Random Forest) vs Boosting (XGBoost, LightGBM, Gradient Boosting)

· 1. Core Concepts & In-Depth Explanation

Supervised Learning predicts outcomes from labeled training pairs:

- **Linear Regression:** Models linear relationships $\hat{y} = w^T x + b$. Regularization prevents overfitting:
- **Lasso (L1):** Adds $\lambda \sum |w_i|$ penalty. Shrinks less important coefficients to absolute zero, performing automated feature selection.
- **Ridge (L2):** Adds $\lambda \sum w_i^2$ penalty. Shrinks coefficients close to zero without zeroing them, handling multicollinearity.
- **Logistic Regression:** Passes linear predictions through the **Sigmoid function** $\sigma(z) = \frac{1}{1 + e^{-z}}$ to constrain outputs into probabilities in range $[0, 1]$.
- **Ensemble Learning:**
- **Random Forest (Bagging):** Trains dozens of deep decision trees independently in parallel on bootstrap samples with random feature subsets. Averages predictions to dramatically reduce variance.
- **Boosting (XGBoost):** Sequentially trains shallow trees, where each subsequent tree focuses on correcting the residual errors of the previous trees. Reduces bias.

· 2. Key Formulas, Rules & Complexity Classes

1. **Sigmoid Activation Function:** $$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{where } z = w^T x + b$$ 2. **Gini Impurity:** $$\text{Gini}(D) = 1 - \sum_{i=1}^C p_i^2$$ 3. **Shannon Entropy:** $$H(D) = -\sum_{i=1}^C p_i \log_2(p_i)$$

· 3. Step-by-Step Solved Examples

Example 2.1: Calculating Gini Impurity for a Binary Split

- **Difficulty / Target Companies:** `Easy | Amazon, TCS Digital`
- **Problem Statement:** A dataset contains 10 samples: 6 Positive and 4 Negative. Calculate the root node Gini Impurity.

Step-by-Step Walkthrough & Logic:

- Step 1: Probability of Positive class $p_1 = 6 / 10 = 0.60$.
- Step 2: Probability of Negative class $p_2 = 4 / 10 = 0.40$.
- Step 3: Gini Impurity $= 1 - (p_1^2 + p_2^2) = 1 - (0.60^2 + 0.40^2)$.
- Step 4: Gini $= 1 - (0.36 + 0.16) = 1 - 0.52 = 0.48$.

Complete Solved Code / Calculation:

```
# Gini calculation:
p_pos = 0.6
p_neg = 0.4
gini = 1.0 - (p_pos**2 + p_neg**2)
print(gini) # 0.48
```

- **Line-by-Line Breakdown & Complexity Analysis:** A pure node (all one class) has Gini = 0; an evenly balanced split has maximum impurity Gini = 0.50.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: Which regularization technique can reduce model feature coefficients to exactly zero, performing automated feature selection?

Options:

- A) Ridge (L2)
- B) Lasso (L1)
- C) Batch Normalization
- D) Dropout

Answer: **B) Lasso (L1)** **Detailed Solution & Reasoning:** The sharp diamond contour of the L1 penalty touches axis intersections at zero, driving non-critical weights to zero.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Amazon)

- **Interview Question:** _Why is Random Forest less prone to overfitting than a single Decision Tree?_
- **Model High-Scoring Answer:** A single deep decision tree has low bias but extremely high variance: it memorizes noise and outlier points in the training set. Random Forest applies **Bootstrap Aggregation (Bagging)** and feature randomness. By training hundreds of distinct trees on independent bootstrap samples and taking their average/majority vote, individual tree variance is divided by the ensemble size, drastically reducing model variance while preserving low bias.
- **Interviewer Follow-Up & Pro Tip:** ? Cite variance reduction through uncorrelated estimator averaging.

CHAPTER 3

Unsupervised Learning & Dimensionality Reduction

Key Principle / Exam Takeaway: Master K-Means clustering, the Elbow Method, Silhouette score, Hierarchical clustering, and Principal Component Analysis (PCA) eigenvalue projections.

Core Curriculum Topics Covered in this Chapter:

- Clustering taxonomy: Centroid-based (K-Means), Density-based (DBSCAN), Hierarchical (Agglomerative)
- K-Means algorithm mechanics: Centroid initialization, assignment, update, convergence
- Optimal K selection: Within-Cluster-Sum-of-Squares (WCSS / Elbow Method) and Silhouette Score
- The Curse of Dimensionality: Distance metrics breakdown in high-dimensional spaces
- Principal Component Analysis (PCA): Covariance matrix, Eigenvalues/Eigenvectors, Explained Variance ratio

· 1. Core Concepts & In-Depth Explanation

Unsupervised Learning uncovers hidden structures in unlabeled data:

- **K-Means Clustering:** Iterative centroid algorithm:

1. Initialize K centroids randomly (or via K-Means++ to maximize initial spread). 2. Assign each sample to the nearest centroid using Euclidean distance. 3. Recompute centroids as the mean of all points assigned to that cluster. 4. Repeat until centroids stabilize.

- **The Curse of Dimensionality:** As dimensions increase, data points become exponentially equidistant from one another, crippling distance-based algorithms.
- **Principal Component Analysis (PCA):** Unsupervised linear dimensionality reduction. Finds orthogonal axes of maximum variance by computing the eigenvectors and eigenvalues of the feature covariance matrix. Projects D-dimensional data onto the top k principal components while retaining maximum informational variance.

· 2. Key Formulas, Rules & Complexity Classes

1. **K-Means Objective (Inertia / WCSS):** $$J = \sum_{j=1}^K \sum_{x_i \in C_j} ||x_i - \mu_j||^2$$ 2. **PCA Eigen-decomposition of Covariance Matrix Σ:** $$\Sigma v_i = \lambda_i v_i \quad \text{where } \lambda_i \text{ represents explained variance along eigenvector } v_i$$

· 3. Step-by-Step Solved Examples

Example 3.1: PCA Explained Variance Calculation

- **Difficulty / Target Companies:** `Easy | Google, Amazon`
- **Problem Statement:** A 3-feature dataset yields eigenvalues $\lambda_1 = 4.0, \lambda_2 = 1.5, \lambda_3 = 0.5$. Calculate the variance explained by the first two principal components.

Step-by-Step Walkthrough & Logic:

- Step 1: Compute total variance: $\sum \lambda = 4.0 + 1.5 + 0.5 = 6.0$.
- Step 2: Component 1 explained variance $= 4.0 / 6.0 = 66.67\%$.
- Step 3: Component 2 explained variance $= 1.5 / 6.0 = 25.00\%$.
- Step 4: Cumulative explained variance of first 2 components $= 66.67\% + 25.00\% = 91.67\%$.

Complete Solved Code / Calculation:

```
// Cumulative Variance:
// (4.0 + 1.5) / 6.0 = 5.5 / 6.0 = 91.67%
// Compressing from 3D to 2D retains over 91.6% of original information!
```

- **Line-by-Line Breakdown & Complexity Analysis:** Demonstrates how top eigenvectors capture the overwhelming majority of information.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: What is the primary vulnerability of K-Means clustering?

Options:

- A) Requires labeled data
- B) Sensitive to outliers, assumes spherical clusters of equal size, and requires specifying K in advance
- C) Only works on 1D data
- D) Too slow on CPU

Answer: **B) Sensitive to outliers, assumes spherical clusters of equal size, and requires specifying K in advance** **Detailed Solution & Reasoning:** Euclidean distance pulls centroids towards extreme outliers, distorting clusters.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Amazon)

- **Interview Question:** _Why must you standardize (scale) features using StandardScaler before applying PCA?_
- **Model High-Scoring Answer:** PCA identifies principal components by maximizing feature variance. Variance is scale-dependent. If one feature is measured in annual income ($50,000 to $200,000, variance $\approx 10^8$) and another in age (20 to 60, variance $\approx 10^2$), PCA will erroneously assign 99.9% of the principal components to income simply due to unit magnitude. Standardizing features to mean 0 and variance 1 ensures all attributes contribute equitably based on true statistical distribution.
- **Interviewer Follow-Up & Pro Tip:** ? Emphasize that variance is scale-sensitive.

CHAPTER 4

Model Evaluation, Cross-Validation & The Bias-Variance Tradeoff

Key Principle / Exam Takeaway: Master the Bias-Variance tradeoff, Confusion Matrix, Precision vs Recall, F1-Score, ROC-AUC, and K-Fold Cross-Validation.

Core Curriculum Topics Covered in this Chapter:

- The Bias-Variance Tradeoff: Underfitting (High Bias) vs Overfitting (High Variance)
- K-Fold Cross-Validation and Stratified K-Fold for imbalanced datasets
- Confusion Matrix: True Positives (TP), False Positives (FP), True Negatives (TN), False Negatives (FN)
- Classification Metrics: Accuracy, Precision, Recall (Sensitivity), Specificity, and F1-Score
- Receiver Operating Characteristic (ROC) curve and Area Under the Curve (AUC)

· 1. Core Concepts & In-Depth Explanation

Evaluating model generalization capability prevents deploying flawed models to production:

- **The Bias-Variance Tradeoff:**
- **High Bias (Underfitting):** Model is overly simplistic (e.g., fitting a linear line to quadratic data). Fails on both training and test data.
- **High Variance (Overfitting):** Model is overly complex (e.g., deep unpruned decision trees). Achieves 100% training accuracy but fails completely on unseen test data.

Classification Metrics for Imbalanced Data: Accuracy is deceptive in fraud detection (if 99% of transactions are legitimate, predicting 'all legitimate' yields 99% accuracy while detecting 0 fraud!).

- **Precision:** Of all predicted positives, how many were truly positive? $\frac{TP}{TP + FP}$. (Prioritize when False Positives are costly, e.g., spam filters).
- **Recall (Sensitivity):** Of all actual positives, how many did the model find? $\frac{TP}{TP + FN}$. (Prioritize when False Negatives are dangerous, e.g., cancer detection, fraud).
- **F1-Score:** Harmonic Mean of Precision and Recall.

· 2. Key Formulas, Rules & Complexity Classes

1. **Classification Metrics:** $$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$ 2. **F1-Score (Harmonic Mean):** $$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Calculating Precision, Recall, and F1-Score from Confusion Matrix

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Infosys`
- **Problem Statement:** A cancer diagnostic model produces: TP = 80, FP = 20, FN = 10, TN = 890. Calculate Precision, Recall, and F1-Score.

Step-by-Step Walkthrough & Logic:

- Step 1: $\text{Precision} = \frac{TP}{TP + FP} = \frac{80}{80 + 20} = \frac{80}{100} = 0.80$ (80%).
- Step 2: $\text{Recall} = \frac{TP}{TP + FN} = \frac{80}{80 + 10} = \frac{80}{90} = 0.889$ (88.9%).
- Step 3: $F_1 = 2 \times \frac{0.80 \times 0.8889}{0.80 + 0.8889} = \frac{1.4222}{1.6889} = 0.842$ (84.2%).

Complete Solved Code / Calculation:

```
// Metric Summary:
// Precision: 80.0%
// Recall:    88.9%
// F1-Score:  84.2%
```

- **Line-by-Line Breakdown & Complexity Analysis:** Harmonic mean penalizes extreme imbalances between precision and recall.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: In a medical diagnosis system detecting a lethal disease, which metric should be prioritized above all others?

Options:

- A) Precision
- B) Recall
- C) Specificity
- D) Training speed

Answer: **B) Recall** **Detailed Solution & Reasoning:** Failing to detect a patient who has the disease (False Negative) can be fatal, so Recall must be maximized to catch all cases.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Google)

- **Interview Question:** _What does an Area Under the ROC Curve (ROC-AUC) of 0.5 vs 1.0 represent?_
- **Model High-Scoring Answer:** The ROC curve plots True Positive Rate (Recall) against False Positive Rate across all possible classification decision thresholds. An **AUC of 1.0** represents a perfect classifier that cleanly separates all positive and negative samples at some threshold. An **AUC of 0.5** represents a worthless model whose discriminative power is no better than random coin-flip guessing. An AUC < 0.5 indicates the model's predictions are inverted.
- **Interviewer Follow-Up & Pro Tip:** ? Cite threshold-independence of ROC-AUC.

CHAPTER 5

Neural Networks & Deep Learning Foundations

Key Principle / Exam Takeaway: Master Perceptrons, Multi-Layer Perceptrons (MLP), Activation functions (ReLU, Sigmoid, Softmax), Forward/Backward pass, and Backpropagation chain rule.

Core Curriculum Topics Covered in this Chapter:

- Biological vs Artificial Neurons: Weights, biases, activation thresholding
- Activation Functions: Sigmoid, Tanh, ReLU (Rectified Linear Unit), Leaky ReLU, Softmax
- Multi-Layer Perceptron (MLP) architecture: Input, Hidden, and Output layers
- Forward Propagation: Matrix vectorization z = Wx + b, a = f(z)
- The Backpropagation Algorithm: Applying the multivariable chain rule to compute weight gradients

· 1. Core Concepts & In-Depth Explanation

Deep Learning models hierarchical representations using layered artificial neural networks.

Activation Functions: Without non-linear activation functions, stacking multiple neural layers collapses mathematically into a single linear regression ($W_2(W_1 x + b_1) + b_2 = W_{net} x + b_{net}$). Non-linear activations enable networks to approximate arbitrary functions:

- **Sigmoid:** Maps $(-\infty, \infty) \to (0, 1)$. Suffers from **Vanishing Gradients** because derivatives saturate near 0 for extreme values.
- **ReLU (Rectified Linear Unit):** $f(z) = \max(0, z)$. Derivative is strictly 1 for $z > 0$. Eliminates vanishing gradients and enables fast computation.
- **Softmax:** Normalizes an unconstrained output logit vector into a valid probability distribution summing to 1.0 for multi-class classification.

Backpropagation: Applies the **Calculus Chain Rule** backwards from the output loss through intermediate hidden layers, calculating partial derivatives $\frac{\partial L}{\partial W}$ to update weights via gradient descent.

· 2. Key Formulas, Rules & Complexity Classes

1. **The Chain Rule for Weight Gradients:** $$\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial W^{[l]}} = \delta^{[l]} \cdot (a^{[l-1]})^T$$ 2. **Softmax Function:** $$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Calculating Softmax Probabilities for Output Logits

- **Difficulty / Target Companies:** `Easy | Google, Amazon`
- **Problem Statement:** A neural network outputs logits $z = [2.0, 1.0, 0.1]$ for 3 classes. Compute the Softmax probability distribution.

Step-by-Step Walkthrough & Logic:

- Step 1: Compute exponentials: $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$.
- Step 2: Sum exponentials: $\sum e^{z_i} = 7.389 + 2.718 + 1.105 = 11.212$.
- Step 3: Divide each by sum: $P_1 = 7.389 / 11.212 = 0.659$ (65.9%); $P_2 = 2.718 / 11.212 = 0.242$ (24.2%); $P_3 = 1.105 / 11.212 = 0.099$ (9.9%).
- Step 4: Sum of probabilities $= 0.659 + 0.242 + 0.099 = 1.000$ (100%).

Complete Solved Code / Calculation:

```
import numpy as np

logits = np.array([2.0, 1.0, 0.1])
exp_z = np.exp(logits)
softmax_probs = exp_z / np.sum(exp_z)
print(np.round(softmax_probs, 3)) # [0.659, 0.242, 0.099]
```

- **Line-by-Line Breakdown & Complexity Analysis:** Converts raw unbounded output activations into calibrated probabilities.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: What is the Vanishing Gradient problem in deep networks using Sigmoid activations?

Options:

- A) Network runs out of memory
- B) Gradients diminish exponentially towards zero as they backpropagate through deep layers, freezing early layer weights from learning
- C) Loss becomes negative
- D) Weights become infinity

Answer: **B) Gradients diminish exponentially towards zero as they backpropagate through deep layers, freezing early layer weights from learning** **Detailed Solution & Reasoning:** The maximum derivative of Sigmoid is 0.25; multiplying fractions across 10 layers yields $(0.25)^{10} pprox 10^{-6}$, vanishing the gradient.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Google)

- **Interview Question:** _What is Dropout and how does it prevent overfitting in deep neural networks?_
- **Model High-Scoring Answer:** Dropout is a regularization technique where during each forward training pass, individual neurons are randomly deactivated ('dropped') with probability p (e.g., 0.5). This prevents neurons from co-adapting (relying excessively on specific neighboring neurons to correct their errors). It forces the network to learn redundant, robust internal representations. At test/inference time, dropout is turned off, and weights are scaled by $(1 - p)$, acting mathematically as an ensemble average over 2^N sub-networks.
- **Interviewer Follow-Up & Pro Tip:** ? Cite breaking co-adaptation and ensemble behavior at test time.

CHAPTER 6

Convolutional Neural Networks & Recurrent Networks

Key Principle / Exam Takeaway: Master CNNs (Convolutions, Kernels, Padding, Stride, Pooling), Image Classification architectures (ResNet), and RNNs (Vanishing gradients, LSTMs, GRUs).

Core Curriculum Topics Covered in this Chapter:

- CNN building blocks: Convolutional filters, Kernels, Stride, Valid vs Same Padding
- Pooling layers: Max Pooling vs Average Pooling for spatial dimension reduction
- Classic CNN architectures: LeNet, VGG, ResNet (Skip/Residual connections)
- Recurrent Neural Networks (RNN): Hidden state persistence for sequential data
- Long Short-Term Memory (LSTM) gates: Forget Gate, Input Gate, Cell State, Output Gate

· 1. Core Concepts & In-Depth Explanation

Specialized architectures handle spatial (images) and temporal (sequences) data:

- **Convolutional Neural Networks (CNN):** Exploits spatial locality in images through weight sharing and translation invariance:
- **Convolution:** Sliders a small kernel matrix (e.g., 3×3) across an image, computing dot products to detect low-level features (edges) in early layers and high-level features (faces) in deep layers.
- **ResNet (Skip Connections):** Adds the input identity to the output of a residual block: $F(x) + x$. This allows gradients to flow directly backwards without vanishing, enabling networks with 150+ layers.
- **Recurrent Neural Networks (RNN):** Processes sequential data (audio, text) by feeding the previous hidden state back into the current step. Standard RNNs suffer from vanishing gradients across long sequences.
- **LSTMs (Long Short-Term Memory):** Solves long-term sequence dependencies via a persistent **Cell State** regulated by three gates:

1. **Forget Gate:** Decides what to discard from cell state (0 to 1). 2. **Input Gate:** Decides what new information to store in cell state. 3. **Output Gate:** Decides what to emit as the hidden state output.

· 2. Key Formulas, Rules & Complexity Classes

1. **CNN Output Feature Map Dimension:** $$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$ Where $W = \text{Input Size}$, $K = \text{Kernel Size}$, $P = \text{Padding}$, $S = \text{Stride}$. 2. **ResNet Residual Block:** $$y = \text{ReLU}(F(x, \{W_i\}) + x)$$

· 3. Step-by-Step Solved Examples

Example 6.1: Calculating CNN Output Dimensions After Convolution

- **Difficulty / Target Companies:** `Easy | Amazon, Google`

- **Problem Statement:** An input image of size $32 \times 32 \times 3$ is passed through a layer with 16 filters of size 5×5, with Stride $S = 1$ and Padding $P = 2$ (Same padding). Find output dimensions.

Step-by-Step Walkthrough & Logic:

- Step 1: Formula: $O = \lfloor (W - K + 2P) / S \rfloor + 1$.
- Step 2: $O = \lfloor (32 - 5 + 2(2)) / 1 \rfloor + 1 = \lfloor (32 - 5 + 4) / 1 \rfloor + 1 = 31 + 1 = 32$.
- Step 3: Depth equals the number of filters $= 16$.
- Step 4: Output feature map is $32 \times 32 \times 16$.

Complete Solved Code / Calculation:

```
// Output Shape:
// (32 - 5 + 2*2)/1 + 1 = 32
// Dimensions: 32 x 32 x 16
```

- **Line-by-Line Breakdown & Complexity Analysis:** Padding of 2 preserves spatial height and width when using a 5x5 filter with stride 1.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: Why are Skip Connections (Residual Connections) in ResNet effective for training very deep networks?

Options:

- A) They compress images
- B) They provide a clean identity gradient highway, allowing gradients to propagate directly to early layers without diminishing
- C) They eliminate training parameters
- D) They replace convolutions

Answer: **B) They provide a clean identity gradient highway, allowing gradients to propagate directly to early layers without diminishing** **Detailed Solution & Reasoning:** The derivative of $F(x) + x$ includes a $+1$ term, ensuring the gradient never vanishes completely.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Amazon)

- **Interview Question:** _What is the function of the Forget Gate in an LSTM network?_
- **Model High-Scoring Answer:** The Forget Gate regulates which information from the previous Cell State (C_{t-1}) should be erased or retained. It takes the previous hidden state (h_{t-1}) and current input (x_t), applies a linear transformation, and outputs values between 0 and 1 via a Sigmoid activation: $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$. A value of 0 means 'completely forget this information', while 1 means 'completely retain this information'.
- **Interviewer Follow-Up & Pro Tip:** ? Cite the element-wise multiplication with the Cell State.

CHAPTER 7

Natural Language Processing & Transformers Architecture

Key Principle / Exam Takeaway: Master Tokenization, Word2Vec embeddings, Self-Attention mechanism, Transformer Encoder-Decoder (Attention is All You Need), BERT vs GPT, and RAG.

Core Curriculum Topics Covered in this Chapter:

- Text preprocessing: Tokenization (BPE, WordPiece), Lemmatization, Stopwords
- Word Embeddings: Dense vectors, Word2Vec (Skip-Gram vs CBOW), Cosine Similarity
- The Transformer Revolution: Attention Is All You Need (Vaswani et al.)
- Scaled Dot-Product Self-Attention: Query (Q), Key (K), Value (V) mechanics
- Encoder-only (BERT) vs Decoder-only (GPT) and Retrieval-Augmented Generation (RAG)

· 1. Core Concepts & In-Depth Explanation

Modern AI is driven by the **Transformer** architecture, which replaces sequential recurrence (RNNs) with parallelized **Self-Attention**:

- **Self-Attention Mechanism:** Allows every word in an input sequence to attend to every other word simultaneously, capturing long-range contextual relationships in $O(1)$ sequential operations.

1. Each word embedding is projected into three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**. 2. Compatibility score between Query i and Key j is computed via dot product: $Q K^T$. 3. Scaled by $\sqrt{d_k}$ (to prevent vanishing gradients in Softmax) and passed through Softmax. 4. Multiplied by Value vector V to compute the contextualized representation.

- **BERT (Encoder):** Bidirectional. Trained on Masked Language Modeling. Optimal for search, classification, and embeddings.
- **GPT (Decoder):** Autoregressive causal attention (masks future tokens). Optimal for text generation.

- **Retrieval-Augmented Generation (RAG):** Combines an external vector database (Qdrant, Pinecone) with an LLM to ground text generation in factual corporate textbooks and documents.

· 2. Key Formulas, Rules & Complexity Classes

1. **Scaled Dot-Product Attention:** $$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$ 2. **Cosine Similarity between Embeddings:** $$\text{Sim}(u, v) = \frac{u \cdot v}{||u|| \times ||v||}$$

· 3. Step-by-Step Solved Examples

Example 7.1: Computing Scaled Dot-Product Attention for 2 Tokens

- **Difficulty / Target Companies:** `Medium | Google, OpenAI, Amazon`
- **Problem Statement:** Given Query $Q = [[1, 0]]$, Keys $K = [[1, 0], [0, 1]]$, Values $V = [[10, 20], [30, 40]]$, and $d_k = 1$. Calculate attention output.

Step-by-Step Walkthrough & Logic:

- Step 1: Compute dot products: $Q K^T = [1(1) + 0(0), 1(0) + 0(1)] = [1, 0]$.
- Step 2: Scale by $\sqrt{d_k} = \sqrt{1} = 1 \implies [1, 0]$.
- Step 3: Apply Softmax: $e^1 = 2.718, e^0 = 1.0$. Sum $= 3.718$. Softmax $= [0.731, 0.269]$.
- Step 4: Multiply by V: Output $= 0.731 \times [10, 20] + 0.269 \times [30, 40] = [7.31 + 8.07, 14.62 + 10.76] = [15.38, 25.38]$.

Complete Solved Code / Calculation:

```
import numpy as np

Q = np.array([[1.0, 0.0]])
K = np.array([[1.0, 0.0], [0.0, 1.0]])
V = np.array([[10.0, 20.0], [30.0, 40.0]])

scores = np.dot(Q, K.T) / np.sqrt(1.0)
weights = np.exp(scores) / np.sum(np.exp(scores))
output = np.dot(weights, V)
print(output) # [[15.38, 25.38]]
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates how Queries extract weighted averages of Values based on Key similarities.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: Why is the dot product $Q K^T$ divided by $\sqrt{d_k}$ in the Scaled Dot-Product Attention formula?

Options:

- A) To save memory
- B) For large projection dimensions d_k, dot products grow large in magnitude, pushing Softmax into regions with extremely tiny gradients; dividing by $\sqrt{d_k}$ stabilizes gradients
- C) To convert matrices to integers
- D) To enforce causality

Answer: **B) For large projection dimensions d_k, dot products grow large in magnitude, pushing Softmax into regions with extremely tiny gradients; dividing by $\sqrt{d_k}$ stabilizes gradients** **Detailed Solution & Reasoning:** Assuming components of Q and K are independent random variables with mean 0 and variance 1, their dot product has variance d_k. Dividing by $\sqrt{d_k}$ scales variance back to 1.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Google)

- **Interview Question:** _What is the difference between BERT and GPT in pre-training objectives and architecture?_

- **Model High-Scoring Answer:** BERT is an **Encoder-only** model that uses **Bidirectional Self-Attention** (every token can attend to tokens on both its left and right). It is pre-trained on Masked Language Modeling (predicting random hidden tokens in a sentence). It excels at comprehension, sentiment analysis, and sentence embeddings. GPT is a **Decoder-only** model that uses **Causal / Masked Self-Attention** (tokens can only attend to previous tokens on their left). It is pre-trained on Next-Token Prediction (autoregressive causal generation). It excels at text generation and multi-step reasoning.

- **Interviewer Follow-Up & Pro Tip:** ? Contrast bidirectional attention (BERT) with causal left-to-right attention (GPT).

CHAPTER 8

Solved AI/ML Placement Interview Case Studies & System Design

Key Principle / Exam Takeaway: Master end-to-end ML System Design: Recommendation Systems (Two-tower retrieval + ranking), Fraud Detection, and LLM RAG architectural pipelines.

Core Curriculum Topics Covered in this Chapter:

- ML System Design Framework: Problem Framing, Data Engineering, Model Selection, Offline/Online Evaluation, Serving Infrastructure
- Case Study 1: E-Commerce Recommendation System (Two-Tower embeddings, Vector Index, Cross-Encoder Ranking)
- Case Study 2: Real-Time Credit Card Fraud Detection (Imbalanced data, SMOTE, low latency inference)
- Case Study 3: Enterprise RAG Architecture (Document chunking, Dense retrieval, Reranking, Prompt grounding)
- Model monitoring in production: Data Drift vs Concept Drift detection (KS-test, PSI)

· 1. Core Concepts & In-Depth Explanation

ML System Design integrates machine learning models into robust, scalable production serving systems:

- **Recommendation System Architecture (Two-Tower Model):**

1. **Candidate Retrieval (Millions -> Hundreds):** Dual encoder neural network embeds User profile into vector u and Item catalog into vector v. Uses Approximate Nearest Neighbor (ANN) search in a Vector DB (Qdrant) via HNSW graphs in < 10ms. 2. **Ranking (Hundreds -> Top 10):** A heavy Cross-Encoder GBDT (XGBoost) or Deep Neural Network scores candidate items against rich real-time context (time of day, current session clicks).

- **Production Monitoring (Drift):**
- **Data Drift (Covariate Shift):** Input distribution $P(X)$ changes while $P(y|X)$ remains constant.
- **Concept Drift:** Relationship between features and target $P(y|X)$ changes over time (e.g., consumer spending habits shifting post-pandemic). Detected using Population Stability Index (PSI).

· 2. Key Formulas, Rules & Complexity Classes

1. **Recommendation System Two-Tower Serving:** $$\text{Score}(u, v) = \text{CosineSimilarity}(\text{UserTower}(u), \text{ItemTower}(v))$$ 2. **Population Stability Index (PSI):** $$\text{PSI} = \sum (A_i - E_i) \times \ln\left(\frac{A_i}{E_i}\right) \quad \text{(PSI } > 0.2 \implies \text{Significant Drift, Trigger Retraining!)}$$

· 3. Step-by-Step Solved Examples

Example 8.1: Designing an Enterprise RAG System Architecture

- **Difficulty / Target Companies:** `Hard | Google, Amazon, Microsoft`

- **Problem Statement:** Design a Retrieval-Augmented Generation (RAG) system for university placement students querying thousands of pages of campus interview textbooks with zero hallucination.

Step-by-Step Walkthrough & Logic:

- Step 1: Document Ingestion: Textbooks parsed, chunked into 500-token blocks with 50-token overlap to preserve semantic context across chunk boundaries.
- Step 2: Vector Embedding: Embed chunks using a domain-tuned embedding model (e.g., text-embedding-3) and index in Qdrant Vector DB with HNSW cosine index.
- Step 3: Retrieval: Student query embedded and queried against Qdrant, retrieving Top 10 candidate chunks in < 15ms.
- Step 4: Reranking: Cross-Encoder reranker selects top 3 most relevant textbook excerpts.
- Step 5: Generation: Grounded prompt constructed: 'Using strictly the provided textbook context, answer the student's question. If context does not contain the answer, reply that it is not covered in syllabus.' LLM generates accurate response.

Complete Solved Code / Calculation:

```
// End-to-End RAG Topology:
// Textbook PDFs -> Chunking -> Vector DB (Qdrant)
// Student Query -> Embedding -> Top-10 Retrieval -> Reranker -> Grounded Prompt -> LLM Output
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees zero hallucination by strictly grounding LLM responses in curated textbook content.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: What is the primary difference between Data Drift and Concept Drift in production ML systems?

Options:

- A) Data Drift affects GPUs; Concept Drift affects CPUs
- B) Data Drift means input feature distribution P(X) changes; Concept Drift means the relationship between features and target P(y|X) changes
- C) They are identical
- D) Data drift only occurs in SQL

Answer: **B) Data Drift means input feature distribution P(X) changes; Concept Drift means the relationship between features and target P(y|X) changes** **Detailed Solution & Reasoning:** Data drift is covariate shift in inputs; concept drift changes the underlying target mapping.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Amazon)

- **Interview Question:** _How do you handle severe class imbalance (e.g. 1 fraudulent transaction per 10,000 legitimate transactions) when training a machine learning model?_

- **Model High-Scoring Answer:** 1) **Resampling:** Apply SMOTE (Synthetic Minority Over-sampling Technique) on the training set to interpolate synthetic fraud samples, or cluster centroids under-sampling on majority class. 2) **Algorithmic Weighting:** Set cost-sensitive class weights (e.g., `scale_pos_weight = 10000` in XGBoost) penalizing false negatives 10,000x more heavily. 3) **Metric Selection:** Never evaluate with Accuracy; use Precision-Recall AUC (PR-AUC) and F1-score. 4) **Anomaly Detection:** Treat as an unsupervised novelty detection problem using Isolation Forests or Autoencoders.

- **Interviewer Follow-Up & Pro Tip:** ? Cite class-weighting in loss functions and PR-AUC over ROC-AUC.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.